[2]:
```python
nums = list(map(int, input("Enter array: ").split()))
target = int(input("Enter target: "))

d = {}
for i, num in enumerate(nums):
    if target - num in d:
        print("Indices:", [d[target - num], i])
        break
    d[num] = i
```

```
Enter array:  2 7 11 15
Enter target:  9
Indices: [0, 1]
```

[3]:
```python
s = input("Enter string: ")

char_set = set()
l = 0
max_len = 0

for r in range(len(s)):
    while s[r] in char_set:
        char_set.remove(s[l])
        l += 1
    char_set.add(s[r])
    max_len = max(max_len, r - l + 1)

print("Length:", max_len)
```

```
Enter string:  abcabcbb
Length: 3
```

[4]:
```
n = int(input("Enter number of intervals: "))
intervals = []

for _ in range(n):
    intervals.append(list(map(int, input().split())))

intervals.sort()
merged = [intervals[0]]

for i in intervals[1:]:
    if merged[-1][1] >= i[0]:
        merged[-1][1] = max(merged[-1][1], i[1])
    else:
        merged.append(i)

print("Merged:", merged)
```

```
Enter number of intervals:  3
 1 3
 2 6
 8 10
Merged: [[1, 6], [8, 10]]
```

[5]:
```
nums = list(map(int, input("Enter array: ").split()))

n = len(nums)
res = [1]*n

left = 1
for i in range(n):
    res[i] = left
    left *= nums[i]

right = 1
for i in range(n-1, -1, -1):
    res[i] *= right
    right *= nums[i]

print("Result:", res)
```

```
Enter array:  1 2 3 4
```

[6]:
```
s = input("Enter parentheses string: ")

stack = []
map = {')':'(', '}':'{', ']':'['}

valid = True
for ch in s:
    if ch in map.values():
        stack.append(ch)
    elif ch in map:
        if not stack or stack[-1] != map[ch]:
            valid = False
            break
        stack.pop()

if valid and not stack:
    print("Valid")
else:
    print("Invalid")
```

```
Enter parentheses string:  ({[]})
Valid
```

[10]:
```python
import heapq

def kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]

print(kth_largest([3,2,1,5,6,4], 2))
```

```
5
```

[12]:
```python
def search_rotated(nums, target):
    left, right = 0, len(nums)-1

    while left <= right:
        mid = (left + right)//2

        if nums[mid] == target:
            return mid

        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid-1
            else:
                left = mid+1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid+1
            else:
                right = mid-1

    return -1

print(search_rotated([4,5,6,7,0,1,2], 0))
```

```
4
```

[13]:
```python
from collections import defaultdict

words = input("Enter words: ").split()

d = defaultdict(list)

for word in words:
    key = ''.join(sorted(word))
    d[key].append(word)

print("Groups:", list(d.values()))
```

```
Enter words:  eat tea ate nat bat
Groups: [['eat', 'tea', 'ate'], ['nat'], ['bat']]
```

[2]:
```
def min_platform(arr, dep):
    arr.sort()
    dep.sort()

    i = j = 0
    platforms = result = 0

    while i < len(arr) and j < len(dep):
        if arr[i] < dep[j]:
            platforms += 1
            result = max(result, platforms)
            i += 1
        else:
            platforms -= 1
            j += 1

    return result

print(min_platform([900,940,950], [910,1200,1120]))
```

2

[4]:
```python
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

def has_cycle(head):
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False

# Example
a = Node(1)
b = Node(2)
c = Node(3)
a.next = b
b.next = c
c.next = a  # cycle

print(has_cycle(a))
```

```
True
```

[5]:
```python
from collections import Counter
import heapq

def top_k(nums, k):
    count = Counter(nums)
    return heapq.nlargest(k, count.keys(), key=count.get)

print(top_k([1,1,1,2,2,3], 2))
```

```
[1, 2]
```

[6]:
```python
def word_break(s, wordDict):
    dp = [False]*(len(s)+1)
    dp[0] = True

    for i in range(1, len(s)+1):
        for word in wordDict:
            if i >= len(word) and dp[i-len(word)] and s[i-len(word):i] == word:
                dp[i] = True

    return dp[len(s)]

print(word_break("applepenapple", ["apple","pen"]))
```

[7]:
```
def longest_palindrome(s):
    res = ""

    for i in range(len(s)):
        # odd
        l = r = i
        while l >= 0 and r < len(s) and s[l] == s[r]:
            res = s[l:r+1]
            l -= 1
            r += 1

        # even
        l, r = i, i+1
        while l >= 0 and r < len(s) and s[l] == s[r]:
            res = s[l:r+1]
            l -= 1
            r += 1

    return res

print(longest_palindrome("babad"))
```

```
d
```

[8]:
```python
def trap(height):
    left, right = 0, len(height)-1
    left_max = right_max = 0
    water = 0

    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water

print(trap([0,1,0,2,1,0,1,3,2,1,2,1]))
```

6

[8]:
```python
def trap(height):
    left, right = 0, len(height)-1
    left_max = right_max = 0
    water = 0

    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water

print(trap([0,1,0,2,1,0,1,3,2,1,2,1]))
```

```
6
```